```
#ifndef COURSE_ROSTER_H
#define COURSE_ROSTER_H

#include <string>
#include <iostream>
#include <stdexcept>

// Date of birth structure
struct Date {
  int day;
  int month;
  int year;
};

// Student record payload
struct Student {
  std::string andrewID;  // Unique Student User ID (e.g., "a_user")
  std::string name;     // Full Student Name (e.g., "Jane Doe")
  Date birthdate;       // Date of Birth
};

// Semantic node structure representing a student entry in the roster chain
struct RosterEntry {
  Student record;    // Student record payload
  RosterEntry* next;  // Pointer to the next entry in the roster
};
```

```
using EntryPtr = RosterEntry*; // Type alias for roster entry pointers

// --- Helper Navigation Routines ---

// Returns a pointer to the tail entry in the roster
RosterEntry* last(EntryPtr* list);

// Returns nullptr representing the past-the-end marker
RosterEntry* end(EntryPtr* list);

// Returns true if the roster head pointer is null or points to an empty list
bool isEmpty(EntryPtr* list);

// Returns a pointer to the entry immediately after position 'w'
RosterEntry* next(EntryPtr w, EntryPtr* list);

// Returns a pointer to the entry immediately before position 'w'
RosterEntry* previous(EntryPtr w, EntryPtr* list);

// --- Core Required Lab Functions ---

// Appends a new student record to the end of the roster list
void insert_student(const Student& student, EntryPtr* list);

// Removes roster entry 'w' and deallocates its heap memory
```

```
void delete_student(EntryPtr w, EntryPtr* list);

// Updates the birthdate for a student matching andrewID; returns true if found
bool update_student_dob(const std::string& andrewID, Date newBirthdate, EntryPtr* list);

// Deallocates all entries in the roster and sets the head pointer to nullptr
RosterEntry* empty(EntryPtr* list);

// Outputs formatted student records from head to tail
void printList(EntryPtr* list);

#endif // COURSE_ROSTER_H
```